DASE 7506 Mini Project 2 Report

Li Yingshu | 3036094514

Objective

This project requires a method to select the best answers among 4 respond sets to a question set given by an LLM.

Setup

The repository provided from the course contains the dataset, prediction function and scoring function. The dataset includes responds and labels. The prediction function calls an empty predict function, which is to be implemented.

Method

To compare different answers, logistic regression (referred to as LR in the following report) classifier is used for this project.
The training process involves two steps. First, the answers are first scored individually with a LR. They are then grouped based on their answer and scored again, using the output of stage 1. The result is a comparator that selects the most preferred answer from the most preferred group. The trained weights are stored and provided during inferring.
By default, only those samples where answers disagree are used for training. Training with all samples results in negligible gain in the model's performance.
Both stages are linear: the supplied representations already summarise a whole trace, the stages can be trained and inspected separately, and inference reduces to a pair of matrix products.

Candidate Scoring
The model takes the embeddings of answers, along with other text characteristics like answer length, as the features for a LR. Higher output score represents higher confidence of correctness. Each candidate is scored on its own, so one question contributes four independent training rows.
The 1,024-dimensional embedding is centred within the question and L2-normalised, so the direction shared by all four candidates, which is the question itself, cancels out and only what is specific to the trace remains. Twelve surface features include length, formatting (####, \boxed, equations, fractions, approximations) and punctuation. Six cross-candidate features describe a candidate against its three siblings: how many of them share its parsed answer, its embedding similarity to the group centroid, to the candidates that agree with it and to those that do not, and the length of its embedding.
A candidate is labelled positive when its parsed final answer equals the reference, which is the rule the scorer itself uses.

Features are standardised and the classifier is fitted with a strong L2 penalty (C = 0.01). Validation accuracy is flat below C = 0.01 (91.35% at 0.001, 0.003 and 0.01), falls to 91.10% at C = 0.1 and to 90.85% at C = 1, so the chosen value sits in a broad optimum rather than on a sharp peak.

Group Comparison

Stage 1 ranks candidates, but the decision is between final answers. Candidates that produce the same results are interchangeable for the scorer, but the correct answer may be from a single candidate. Therefore, candidates are merged into groups by parsed answer, and stage 2compares between groups for the best one.

The comparator is a second LR, trained on differences between two groups of the same question. It sees the mean, maximum and minimum stage-1 score inside the group, the group's support (the number of included candidates), its internal cohesion and its similarity to the question centroid, together with averaged text statistics. The target is the difference between a correct group and a wrong one, so one weight vector scores any group; the highest-scoring group wins and the best-scoring candidate inside it is returned.

The comparator is trained on out-of-fold stage-1 scores from five folds. This is to prevent the stage-1 score of a training question from dominating other group features. Its own accuracy is estimated by refitting it on four folds and scoring the fifth, which gives around 91.36.

Training and Inference

Only questions whose four candidates disagree are used for training; that is 2513 of the 4755 samples (52.8% of the database). The remaining groups offer nothing to compare and produces very little improvements for the result.

Fitting both stages, including the five folds and the comparator cross-validation, takes about 12 seconds on the development laptop (AMD Ryzen 7 7840H using four threads). The checkpoint stores the two weight vectors and their standardisation statistics in 26 KiB.

At inference the whole test bank is processed as one batch: a fresh process predicts the 467 questions in 0.33 seconds, 0.85 seconds of wall clock including imports and data loading, and the independent scorer adds 0.03 seconds. The 30-second budget is not binding for this method, so the remaining room is available to a more expensive method rather than to a faster one.

Results & Analysis

All numbers below come from the official scorer on the fixed splits. Validation drives the analysis because the method was selected on it; the test column is the reported score.

Selection rule	Validation	Test
Voting only	86.22	69.16
Stage-1 verifier	91.35	80.73
Stage 1 + comparator	91.85	82.87

Individual scoring carries most of the gain

As seen in the table, replacing voting with the verifier raises validation accuracy from 86.22% to 91.35% and test accuracy from 69.16% to 80.73%. It is the largest single improvement in the report, and it costs one matrix product.

Voting is a strong baseline only where candidates repeat an answer. When all four candidates end in the same number the choice cannot be wrong, which is the case for 443 of the 798 validation questions. No test group is unanimous, yet a majority answer is still correct in 235 of the 303 two-group questions, which is why voting keeps 69.16% on that bank. The verifier adds 54 questions over voting on test, 34 of them two-group questions where the majority answer is plausible but wrong.

Group comparison optimises occasions where answers compete

The comparator improves both splits by a modest margin: 0.50 points on validation and 2.14 points on test. The difference is explained by the composition of the banks, since more than half of the validation questions have a single answer group, where the stage cannot change anything.

On test the comparator changes the selected candidate for 23 of 467 questions and gains 10 correct answers net; on validation it changes 13 and gains 4. The gain is concentrated where several answers compete: accuracy rises from 69.05% to 73.81% with three answer groups and from 55.26% to 65.79% with four, while the two-group case is left untouched at 88.78%. Where one answer already dominates, the stage-1 score settles the question and the comparator usually confirms it.

Embeddings carry most of the signal

The second table removes whole feature blocks and retrains stage 1, so each row is a complete method rather than a reweighting of the full one.

Features	Validation accuracy
Embeddings only (1,024)	90.73
Surface text only (12)	81.70
Cross-candidate only (6)	86.09
Embeddings + surface (1,036)	90.85
All features (1,042)	91.35

The embeddings alone reach 90.73%, within 0.62 points of the full model, so most of the accuracy is already in the fixed representation. Surface features alone reach 81.70%, below voting: length, equation density and formatting say something about style, but not enough about correctness. Cross-candidate features alone reach 86.09%, which confirms that agreement carries real information, and they are worth more on top of the embeddings (0.50 points) than the surface block is (0.12 points). The blocks are complementary, but not equally so.

Validation is easier than the test bank

Validation accuracy is 91.85% and test accuracy is 82.87%, and most of that gap belongs to the banks rather than to the method. Every rule scores 100% on the 443 validation questions whose candidates parse to a single answer: the group is

guaranteed at least one correct candidate, so the one answer on offer must be that correct answer. The test bank contains no such question.
Restricting validation to the 355 questions with at least two answer groups removes the gap: voting 69.01% against 69.16% on test, the verifier 80.56% against 80.73%, and the full method 81.69% against 82.87%. The method transfers well, and 81.69% is the more honest estimate of what it does when every question needs a decision.

The remaining errors are questions with four different answers
Accuracy falls steadily with the number of answer groups. On test it is 88.78% with two groups, 73.81% with three and 65.79% with four; validation shows the same ordering (100.00%, 88.11%, 72.22% and 60.00%). When all four candidates end in different numbers the model has to separate four confident traces with nothing to cross-check, and it is right about two thirds of the time.
Group comparison recovers part of that case (55.26% to 65.79% on test), which suggests that cross-candidate evidence still helps when no two candidates agree, but the evidence in four contradictory traces is thin.
More training questions do not help either. Training on all 4,755 questions instead of the 2,513 informative ones moves validation accuracy from 91.35% to 91.23%, a difference inside fold noise, so the simpler training set is kept.

Conclusion

The method treats selection as two cheap linear decisions: score each candidate, then compare the answer groups the candidates form. It reaches 82.87% on the test bank with a 26 KiB checkpoint and less than a second of CPU time. The ablation shows that almost all of the accuracy comes from the supplied embeddings rather than from surface style, and that the group stage earns its place only when several answers compete.
What the method cannot see is the reasoning itself. The representation is a single whole-trace vector, so intermediate steps are never inspected, and the questions that fail are those where four traces contradict each other. The 30-second budget allows roughly thirty times the current computation, which would be enough for a model over individual steps rather than whole traces; testing it would show whether the missing 17 points are a limit of the representation or of the task.

AI Usage Declaration

Code development is assisted by DeepSeek-V4.1-Flash. The introduction of the group comparator is proposed entirely by the LLM. DeepSeek also helps paraphrased this report.

References

[1] C. R. Harris *et al.*, 'Array programming with NumPy', *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.

[2] F. Pedregosa *et al.*, 'Scikit-learn: Machine Learning in Python', *J. Mach. Learn. Res.*, vol. 12, no. 85, pp. 2825–2830, 2011.